Этот конспект сгенерирован с помощью AI. Система может допускать ошибки в формулах, вычислениях и специфической терминологии. Пожалуйста, относитесь с понимаем и проверяйте конспект!

Обязательное посещение и самостоятельное обучение

Концепция: Программирование — это не просто знание синтаксиса языка, а комплексная дисциплина, включающая алгоритмы, структуры данных, прикладные библиотеки и творческое (ремесленное) мышление.

Алгоритм: Обучение программированию должно охватывать три уровня:
1. Синтаксис конкретного языка (`Python`, `C++`) — достаточен для базового понимания, но не делает программистом.
2. Алгоритмы и структуры данных — основа эффективности и решения задач.
3. Работа с прикладными библиотеками (`PyTorch`, `FastAPI`, `PostgreSQL`) — ключ к реальному применению в проектах.

Паттерн: Программирование — это «искусство»: программист создаёт текст (код), который становится интеллектуальной собственностью, передаваемой компании при трудоустройстве. Это аналог писательской деятельности с юридическими последствиями по лицензированию.

Программист обязан самостоятельно осваивать пользовательские навыки (`Linux`, `Photoshop`, сетевое администрирование) и системные технологии — кафедра информатики МФТИ фокусируется исключительно на обучении **программированию** как профессиональному навыку, а не на смежных IT-навигациях.

$$L = -\sum y \log \hat{y}$$

$O(n \log n)$

Программирование — это не только синтаксис и алгоритмы, но и совокупность навыков, которые формируют профессионала. Ключевые составляющие: **алгоритмы и структуры данных**, **прикладные библиотеки**, **практика программирования**

(практические приёмы), **дизайн ПО (архитектура приложения)**, **групповая работа** и **ремесло/искусство написания кода**.

Алгоритм:
$O(n \log n)$ — оптимальная сложность для сортировки слиянием

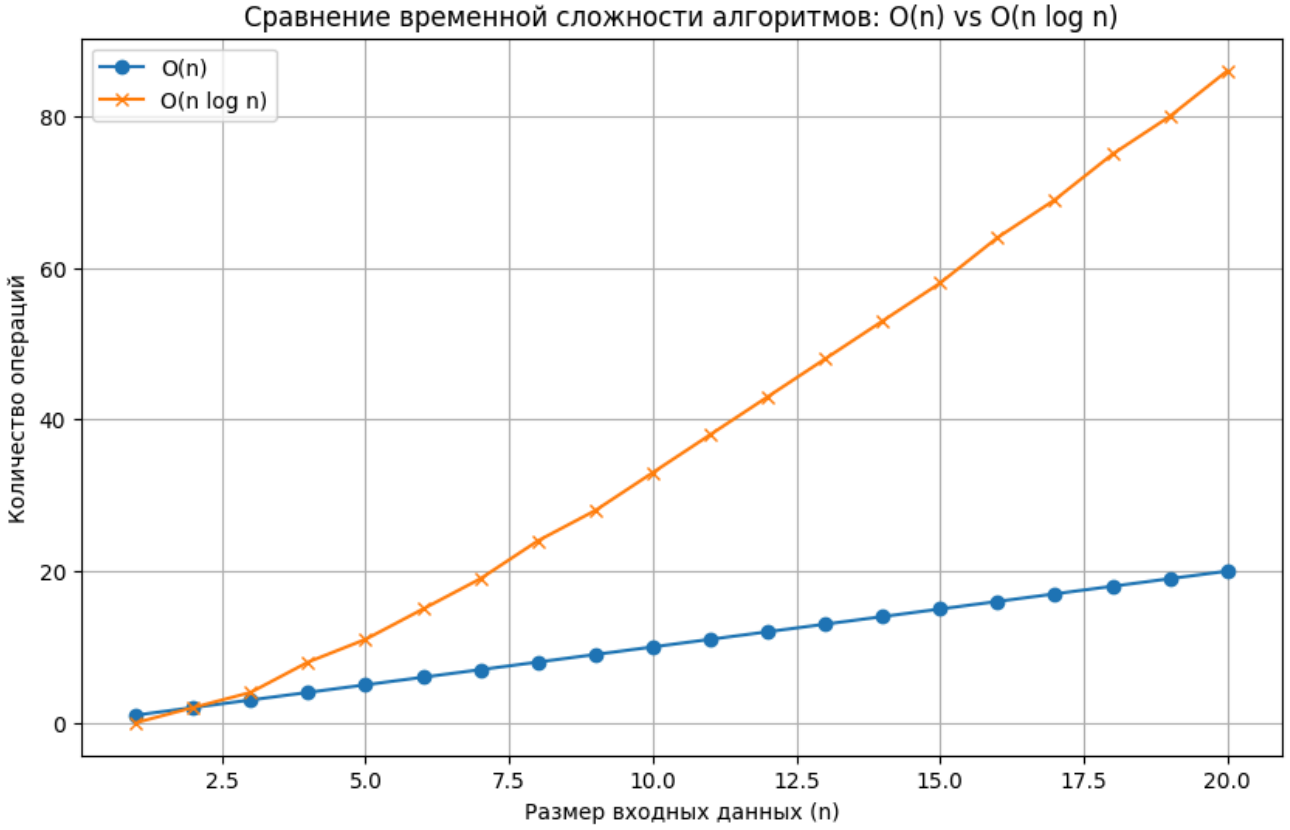
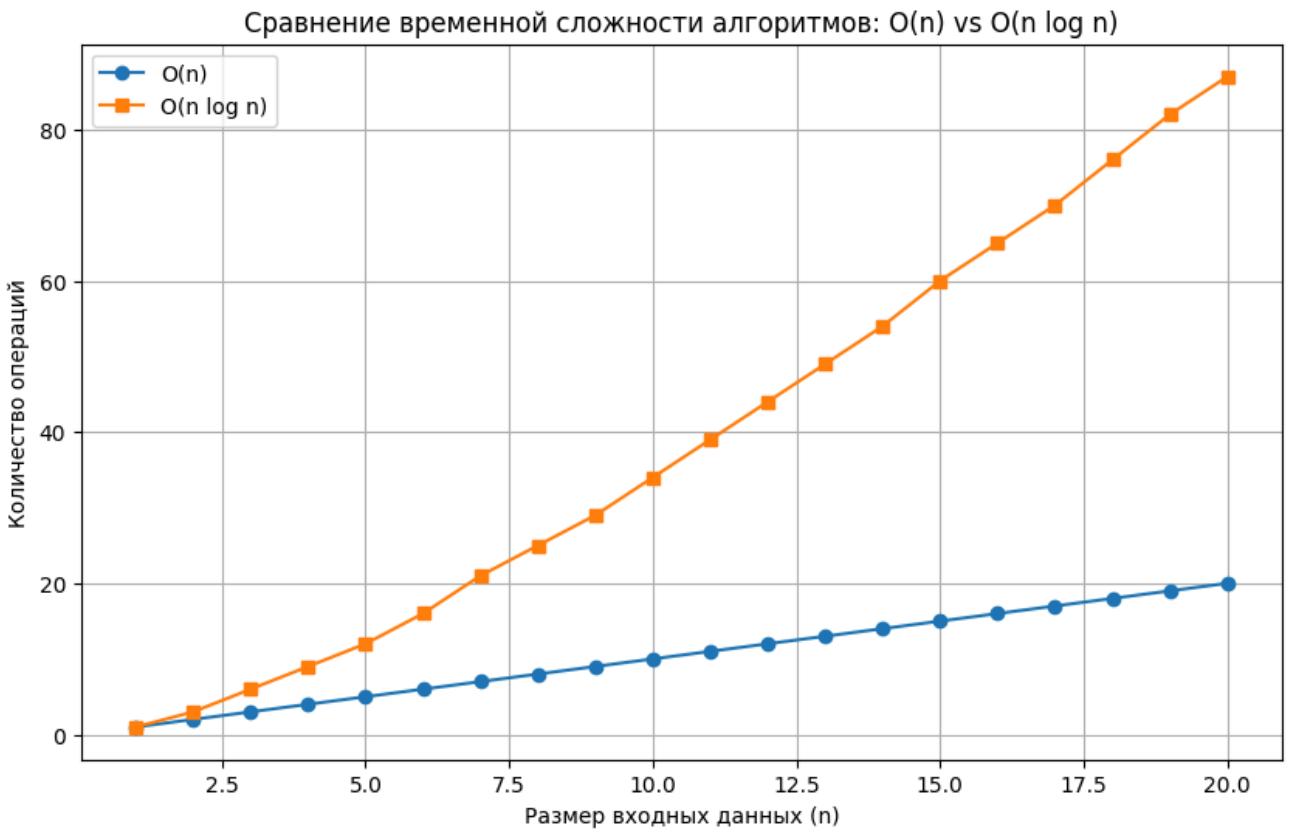


График роста сложности O n log n на практике

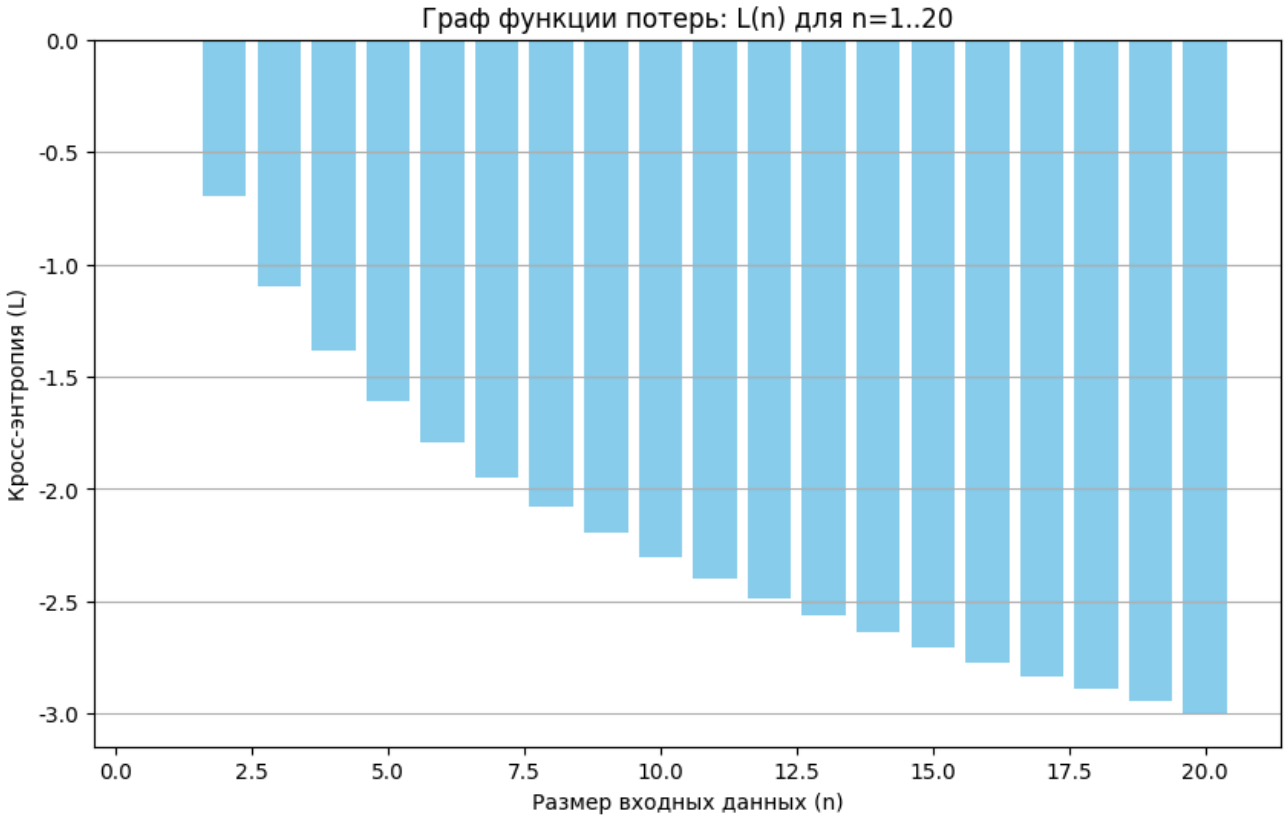


Сравнение временной сложности алгоритмов O n vs O n log n

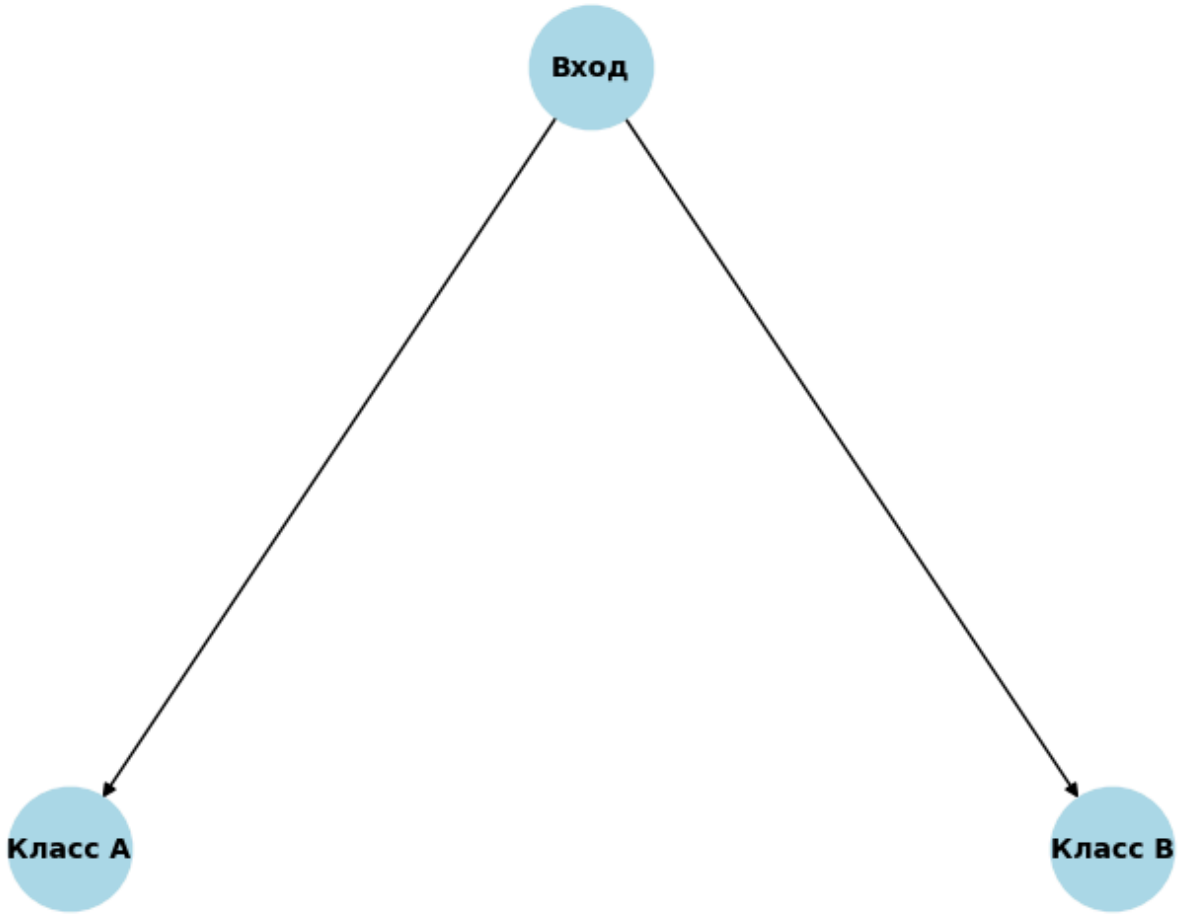
.

$$L = -\sum y \log \hat{y}$$

— кросс-энтропия как функция потерь в задачах классификации



Граф функции потерь L n для n 1 20



Дерево решений простая схема дерева классификации 3 узла

.

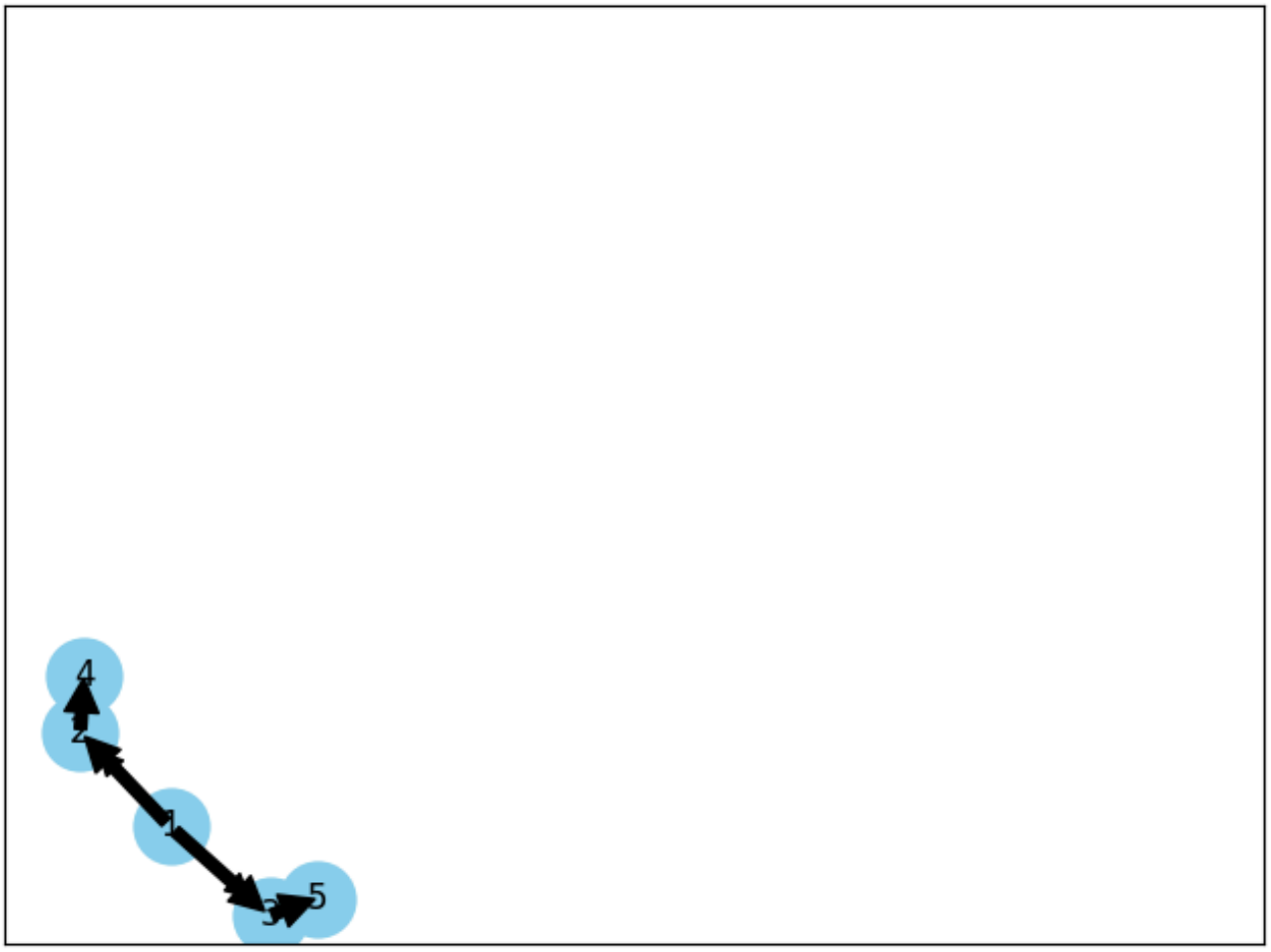
Паттерн:
Архитектура ПО — аналог проектирования здания: программист уровня «рабочий» реализует модули, а специалист по дизайну (архитектор) определяет целостную структуру системы. Разница между ними — масштаб мышления и ответственность за долгосрочную совместимость решений.

Концепция:
Программист — писатель, чьи тексты (коды) становятся интеллектуальной собственностью компании при трудоустройстве. Это юридически закреплённые отношения, аналогичные авторским договорам в издательской сфере.

Паттерн:
Групповая работа — обязательный элемент создания крупных программных продуктов. Без координации, систем трекинга задач и инструментов совместной разработки (например, Jira, Git) масштабируемость проекта становится невозможной.

Пример: строительство Эйфелевой башни без команды — теоретически возможно, но практически нереализуемо.

Концепция:
Язык программирования — инструмент, который меняется быстро. В курсе используется **Python 3**, актуальная версия

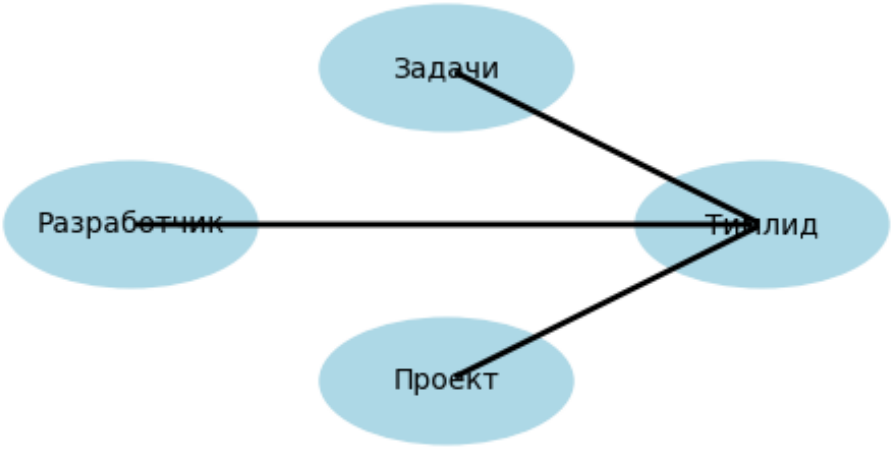


Простое дерево структур данных бинарное дерево поиска BST из 5 узлов

на момент обучения. Акцент делается на синтаксис и логику, а не на устаревшие или специализированные библиотеки (например, MFC, .NET). Прикладные технологии рассматриваются как дополнительные навыки, но не основная цель курса.

Концепция:
Практика программирования — навык, а не знание. Умение применять лучшие практики (по МакКоннеллу и другим авторитетам) — ключевой критерий профессионализма. Без этого даже полный синтаксический охват языка не гарантирует востребованность на рынке.

Паттерн:
Разделение ролей: рядовой программист реализует функции «под ключ», тимлид или генеральный разработчик отвечает за архитектуру



Архитектурная схема разделения ролей в разработке ПО турист

, стратегию и долгосрочную жизнеспособность системы. Это аналог туриста и руководителя группы — один двигается по маршруту, другой определяет маршрут.

Языки программирования, как и исторические события, делятся на временные и вечные. Современный Python 3 — универсальный язык, который мы используем в курсе, но акцент не на синтаксисе, а на фундаментальных принципах программирования. Прикладные библиотеки (вроде графических интерфейсов или внешних API) меняются со временем — их заучивание бессмысленно. Важно уметь **видеть** суть: алгоритмы, структуры данных, парадигмы — то, что остаётся актуальным десятилетиями.

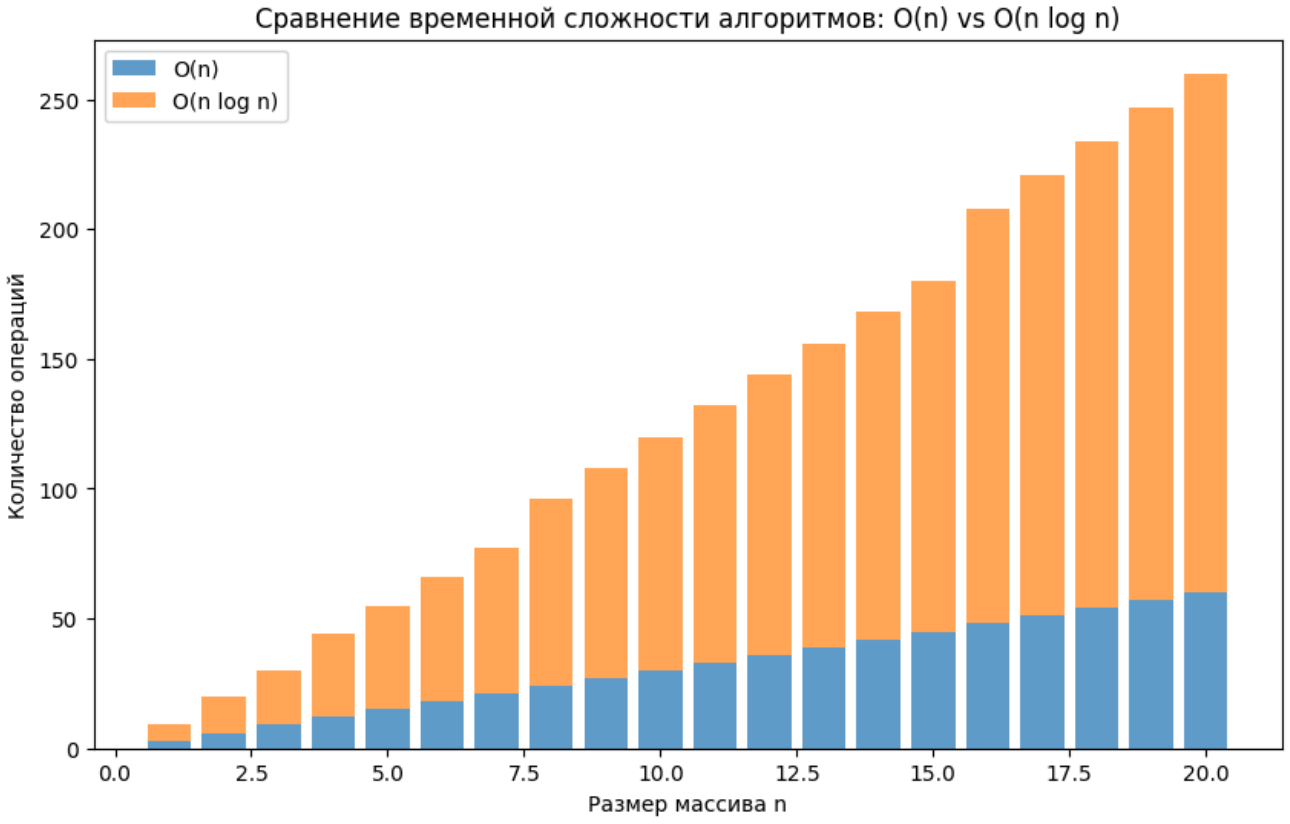
Концентрация курса — на алгоритмах и структурах данных. Групповые проекты в этом семестре не предусмотрены; практика программирования будет развиваться через лабораторные работы, но без обязательного взаимодействия в команде. ООП и другие продвинутые темы (включая функциональное программирование) будут вводиться постепенно, «незаметно» — как естественное расширение мышления, а не как отдельная дисциплина.

Концепция:
Курс состоит из лекций и лабораторных работ. Лекции — обязательный костяк

программы: без них невозможно понять направление курса. Лабораторные развивают практические навыки через решение задач. Посещение лекций **обязательно**, хотя контроль посещаемости не ведётся — отсутствие на занятиях приведёт к потере понимания материала.

Алгоритм:

1. Изучить синтаксис Python 3 (на уровне «вход в океан»).
2. Решать задачи, выходящие за рамки ЕГЭ по информатике.
3. Постепенно осваивать алгоритмы и структуры данных



Сравнение временной сложности алгоритмов O n vs O n log n

.

4. Функциональные и объектно-ориентированные подходы вводятся как естественные расширения, а не как отдельные темы.

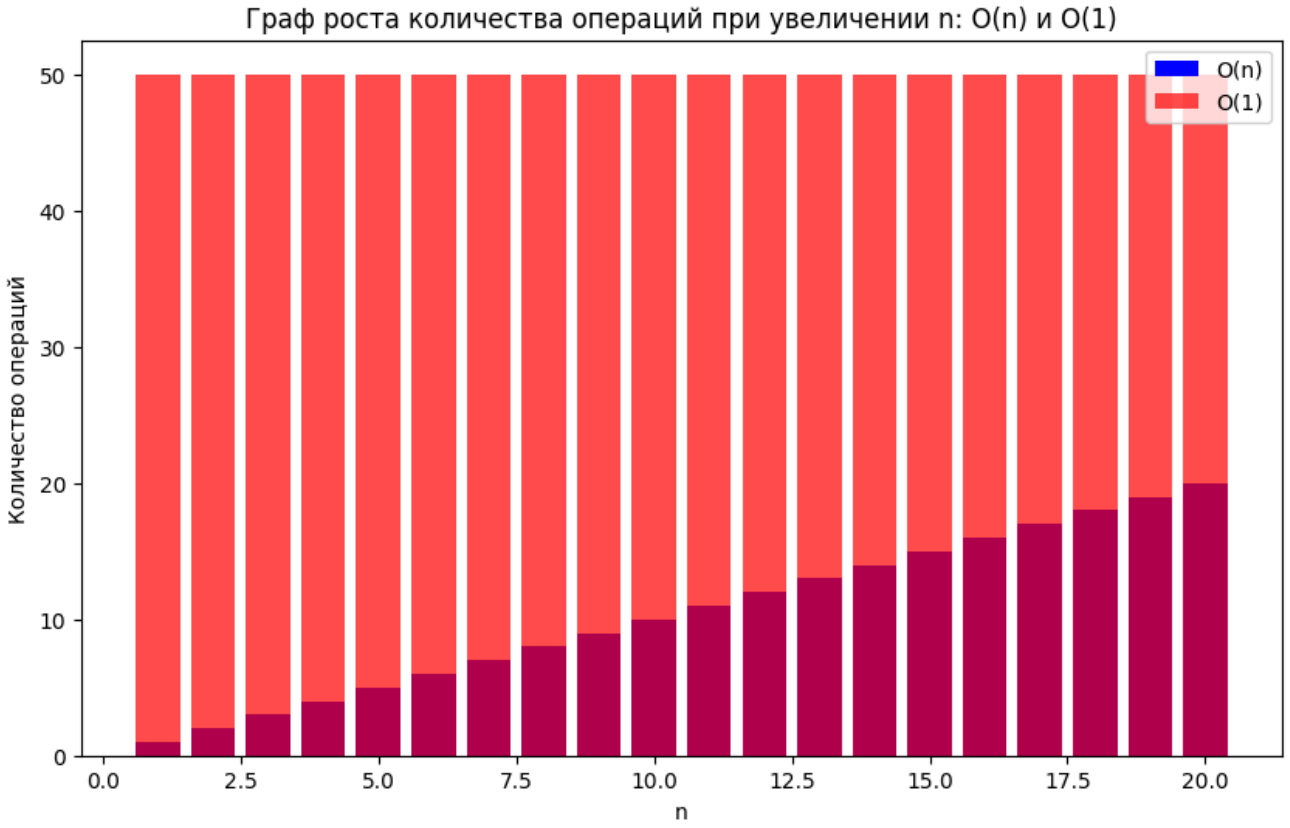
Паттерн:

Разделение ролей: на первом курсе акцент — на индивидуальном освоении базовых конструкций; групповые проекты и архитектура систем (включая тимлидов) — только во втором семестре. Это аналог «туриста» и «руководителя группы»: один осваивает маршрут, другой его определяет.

Лабораторные работы доступны в задавальнике. Лекции будут записаны — но **отсутствие записи не компенсирует пропуск лекций**. Изучение материала по записям после пропущенного занятия неэффективно.

Литература (включая Марк Саммерфилд) — не требуется: она про синтаксис, а он нам сейчас не нужен. Язык Python — «лёгкий вход в океан», но глубина — в алгоритмах и парадигмах.

Цель курса:
Уметь писать сложные алгоритмы на простых конструкциях языка



Граф роста количества операций при увеличении n O n и O 1

, понимать их структуру и развитие — без зависимости от конкретных библиотек или версий.

Простейшая программа на Python — пустой файл, который является корректной программой и ничего не делает. Более сложная программа содержит вызов функции `print`, например:

```
print()
```

— это корректная программа, которая выводит пустую строку (перевод каретки). Следующая версия:

```
print("Hello World")
```

— первая осмысленная программа на Python, выводящая текст.

Для хранения данных в памяти используется механизм именования значений. Например:

```
x = "hello world"
print(x)
```

эквивалентно по результату `print("hello world")`, но позволяет повторно использовать значение под именем `x`.

Тип значения можно получить с помощью функции `type()`:

```
x = "hello world"
print(type(x))  # <class 'str'>
```

Имена (переменные) необходимы, потому что Python автоматически освобождает память от объектов, на которые нет ссылок. После использования значения переменная может быть удалена сборщиком мусора — как студент, сдавший экзамен и забывший материал. Объект строки `"hello world"` больше не нужен программе — он будет удалён из памяти, даже если ранее был доступен через имя `x`.

x — это имя, а не значение. Есть значения (объекты), которые лежат в памяти, и есть имена, ссылающиеся на них. Например, строка `"hello world"` — объект строки типа `str`. До присваивания у него нет имени, но после `x = "hello world"` появляется ссылка: переменная `x` указывает на этот объект.

Тип объекта можно проверить через `type(x)`. В Python тип значения не привязан к имени жёстко — имя лишь ссылается на объект. После использования значение может быть удалено сборщиком мусора, если на него больше нет ссылок. Это аналогично студенту, сдавшему экзамен и забывшему материал: объект "выкидывается" из памяти.

Оператор присваивания `=` в Python **не копирует** значение — он создаёт ссылку. Если `x = 1 + 2 * 3`, интерпретатор сначала вычислит выражение (создав временные

объекты `1`, `2`, `3`), затем создаст объект `9` типа `int`. После этого имя `x` связывается с этим новым объектом, а старые ссылки теряются — их уничтожит сборщик мусора.

В отличие от языков со строгой статической типизацией (например, Pascal или C++), в Python можно присваивать значения разных типов одной переменной:

```
x = "hello world"
print(type(x))  # <class 'str'>
x = 9
print(type(x))  # <class 'int'>
```

Это возможно, потому что `x` — не типизированная ячейка памяти, а **ссылка**, которая может указывать на любой объект.

Алгоритм: обмен значений двух переменных без третьей.
В языках с копированием (C++, Pascal) требуется временная переменная:

```
a = 2
b = 5
tmp = a
a = b
b = tmp
```

В Python, где `=` — это **ссылка**, можно использовать "обмен по ссылке":

```
a = 2
b = 5
a, b = b, a
```

Это работает за счёт одновременного создания новых объектов и переназначения ссылок. Временная переменная не нужна — имена `tmp` не создаётся, память не засоряется.

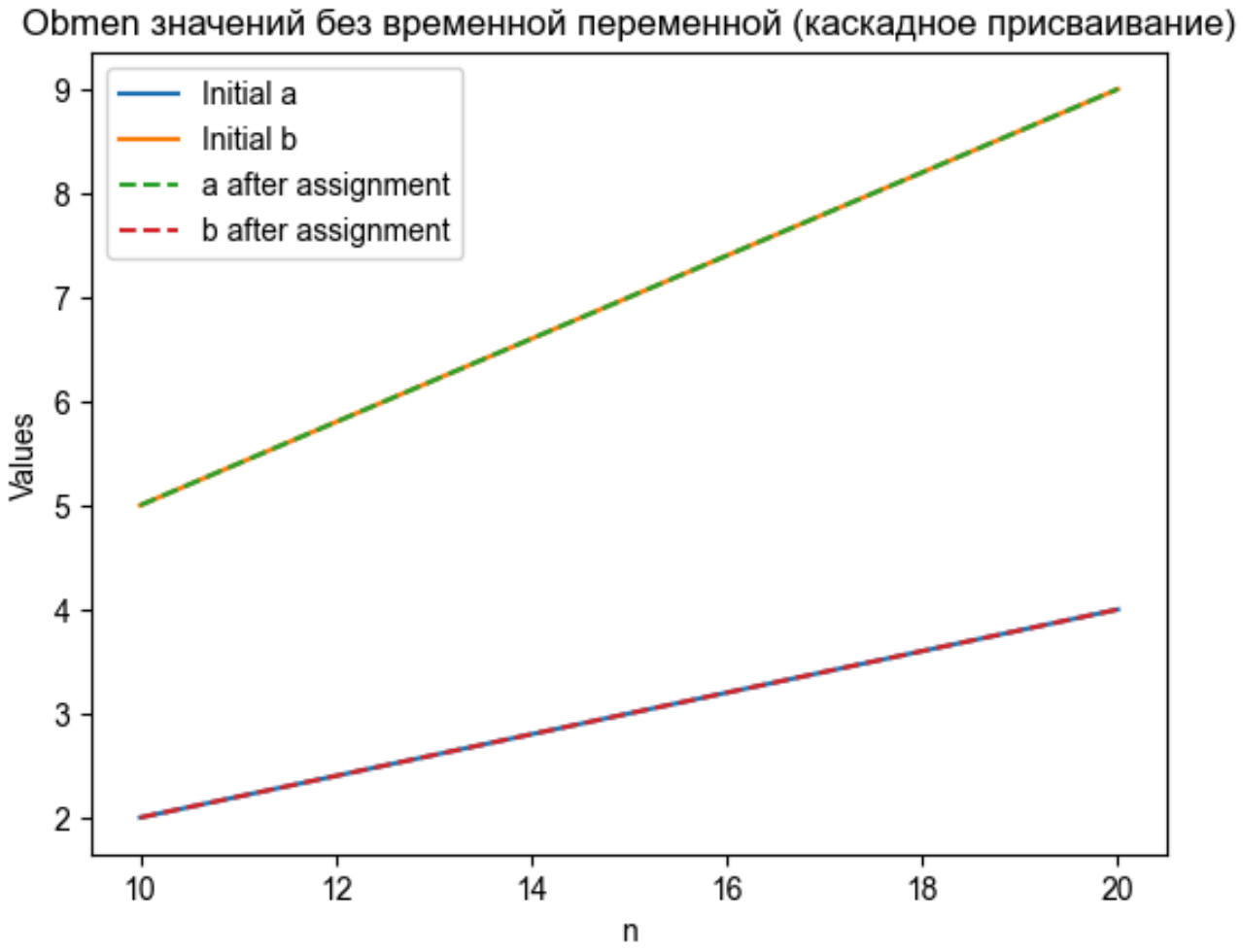
Ключевое отличие: в Python **нет копирования при присваивании**, только связывание имён со значениями. Это упрощает код, но требует понимания модели управления памятью через ссылки и сборщик мусора.

В Python обмен значений между переменными `a` и `b` не требует временной переменной благодаря **отсутствию копирования при присваивании** — имена

просто перенаправляются на объекты. Вместо трёх строк с `tmp = a`, `a = b`, `b = tmp` можно использовать одну:

```
a, b = b, a
```

Это работает за счёт создания временного кортежа `(b, a)` и распаковки его в `a` и `b`. Такой приём называется **каскадным присваиванием** — все имена слева получают значения из правой части по порядку



Обмен значений без временной переменной каскадное присваивание

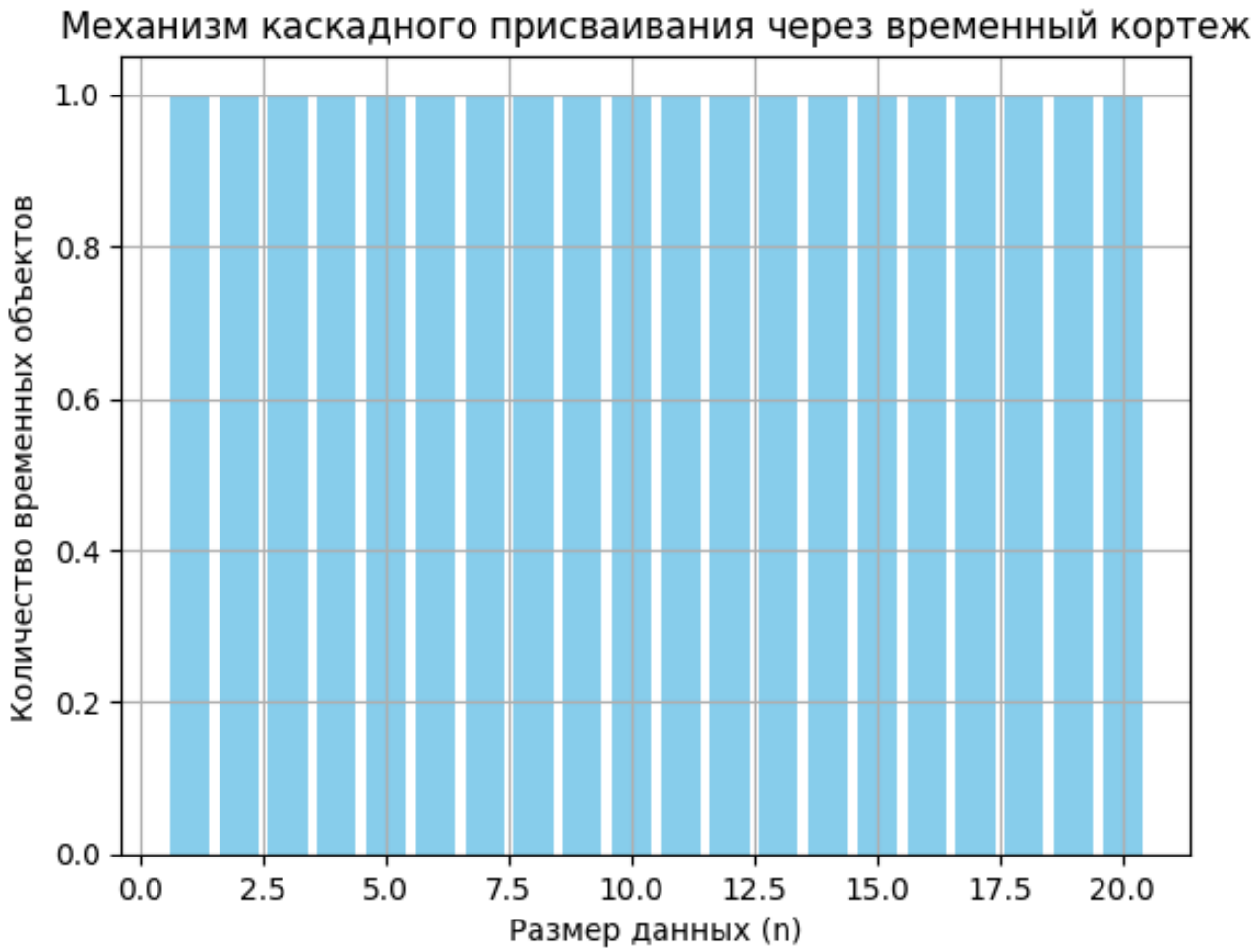
.

Ключевое отличие от C++: в Python нет копирования объектов при присваивании — только связывание имён со ссылками на объекты. Это упрощает код, но требует понимания модели управления памятью через ссылки и сборщик мусора.

Также важно: **неизменяемые типы** (`int`, `float`) нельзя изменить по ссылке — присваивание `x = 5` после `x = 2` создаёт новый объект, а старое значение остаётся в памяти до сбора мусора. Изменить содержимое объекта невозможно — только пересвязывание имени на другой объект.

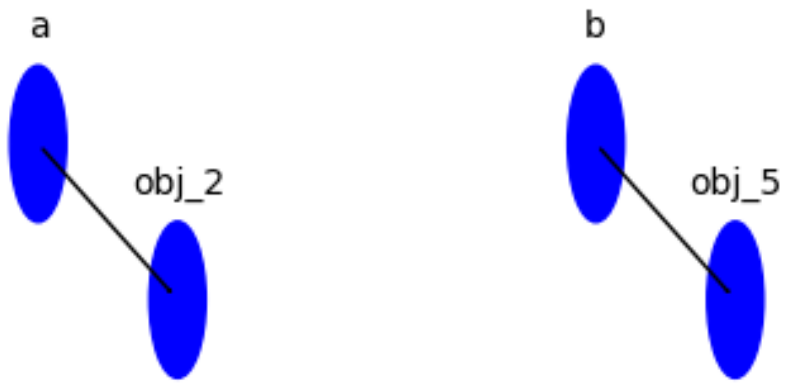
Концепция:
- В Python обмен переменных без временной переменной возможен через каскадное присваивание: `a, b = b, a`.
- Это работает за счёт создания и распаковки временного кортежа `(b, a)`.
- Присваивание не копирует объекты — только перенаправляет имена на существующие ссылки.
- Изменение неизменяемых объектов (например, `int`) невозможно — присваивание создаёт новый объект, старый остаётся в памяти до сбора мусора.

Каскадное присваивание `a, b = b, a` работает за счёт создания временного кортежа `(b, a)` и распаковки его в переменные слева. Присваивание не копирует объекты — только перенаправляет имена на существующие ссылки



Механизм каскадного присваивания через временный кортеж

. Изменить содержимое неизменяемого объекта (например, `int`) невозможно: присваивание создаёт новый объект, старый остаётся в памяти до сбора мусора



Модель управления памятью через ссылки и сборщик мусора

.

Множественное присваивание:
`x, y, z = 1, 2, 3` — это не каскадное, а настоящее множественное присваивание. Оно использует кортеж значений и распаковывает его по позициям в кортеж переменных. Каждое имя получает своё значение без промежуточных копий.

Обмен значений:
`a, b = b, a` — стандартный способ обмена двух переменных без третьей. На самом деле это не обмен "на месте", а присваивание через временный кортеж из ссылок. Объекты не копируются — только перенаправляются ссылки. После вычисления выражения кортеж уничтожается.

Арифметические операции в Python:

- **Возведение в степень:** `x ** y`
  Приоритет: самый высокий среди бинарных операций.
  Поддерживает дробные степени: `3 ** 0.5` — квадратный корень из 3.
  Ассоциативность: справа налево: `(a ** b) ** c`, а не `a ** (b ** c)`.

- **Унарный минус:** `-x` — более высокий приоритет, чем бинарные операции.
  Унарный плюс `+x` — нейтрален, не влияет на значение.

- **Умножение и деление:** одинаковый приоритет.
  Умножение: `x * y`
  Деление: `x / y` — всегда возвращает число с плавающей точкой (`float`), даже если результат целый (в отличие от C или Pascal).
  Пример: `2 / 3 → 0.666...` (тип `float`).

- **Целочисленное деление:** `//` — не упоминается в транскрипте, но логически следует после `/`.

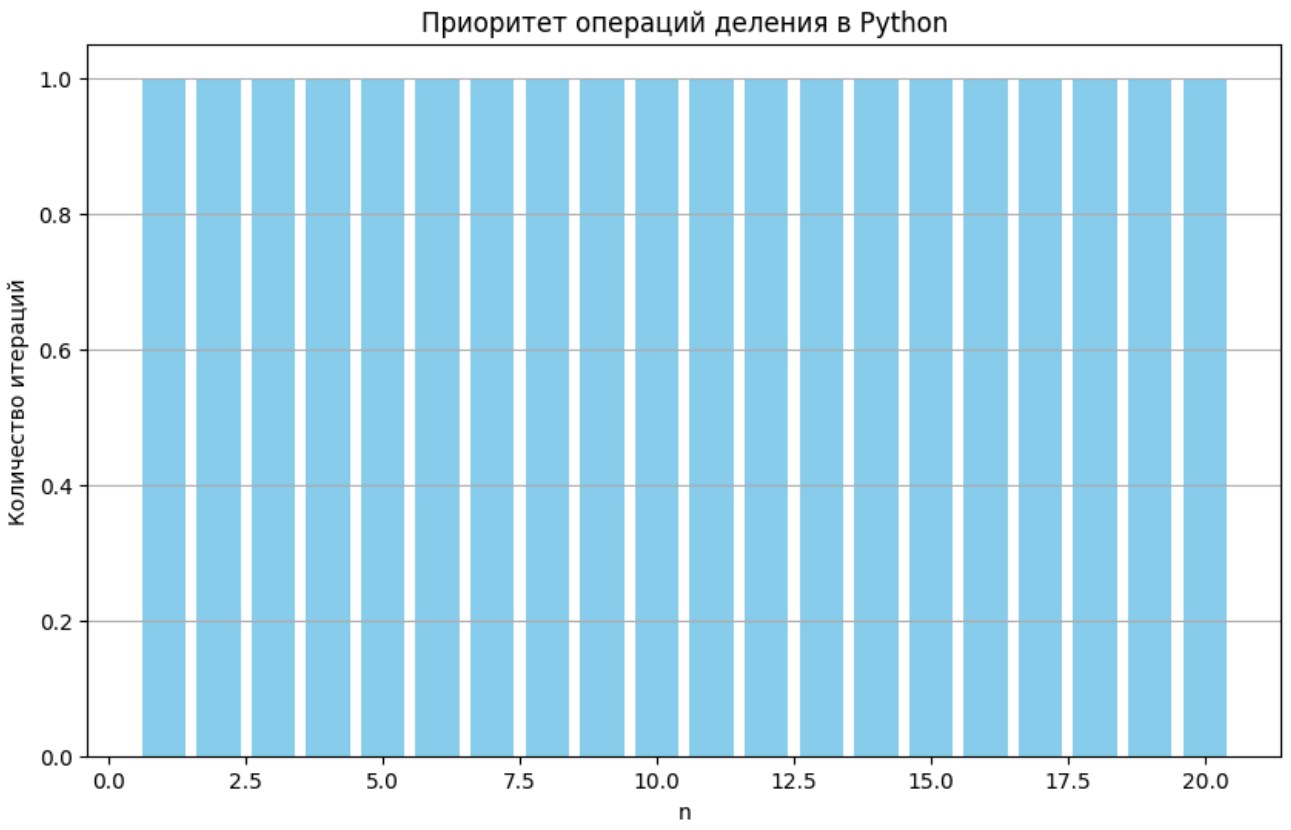
- **Остаток от деления:** `%` — также не упомянут, но входит в стандартный набор.

$$x^y = x \cdot x \cdots x \quad (y \text{ раз})$$

$$\sqrt{x} = x^{0.5}$$

$$a^{b^c} = a^{(b^c)} \quad (\text{в Python: } a**(b**c))$$

Унарный минус имеет приоритет выше умножения и деления.
Операция возведения в степень `**` вычисляется справа налево: `a ** b ** c` эквивалентно `a ** (b ** c)`. Скобки не требуются — порядок определён семантикой оператора.
Умножение (`*`) и деление (`/`, `//`, `%`) имеют одинаковый приоритет и выполняются слева направо



Приоритет операций деления в Python

при отсутствии скобок.
Целочисленное деление: `x // y` — результат без остатка (аналог Pascal `div`).
Остаток от деления: `x % y` — аналог Pascal `mod`.
Пример: `16 // 3 == 5`, `16 % 3 == 1`.

В Python деление с отбрасыванием дробной части (`//`) и остаток от деления (`%`) реализуются через операторы `/` (обычное деление) и `//` (целочисленное), а также `%` (остаток). Целочисленное деление эквивалентно Pascal-операции `div`, например:

```
16 // 3 == 5
```

Остаток — аналог `mod`:

```
16 % 3 == 1
```

Все четыре операции (`/`, `//`, `%`, `/`) имеют **одинаковый приоритет** и выполняются **слева направо**, если скобки отсутствуют. Это означает, что выражение вида:

```
a / b * c
```

интерпретируется как `(a / b) * c`, а не `a / (b * c)`.

Для отрицательных чисел поведение `//` и `%` соответствует **математическому округлению вниз** (toward negative infinity), что отличается от Pascal, где результат зависит от реализации. Например:

```
-12 // 10 == -2
-12 % 10 == 8
```

Это следует из правила:

$$x = (x//y) \cdot y + (x\%y)$$

при любом знаке `x` и `y`.

Такой подход обеспечивает согласованность в вычислениях по модулю, включая использование групп вычетов (например, $\mathbb{Z}_n$), что важно для криптографии.

Цикл `while` — это цикл с предусловием, где проверка условия выполняется перед каждым шагом. Синтаксис:

```
while условие:
    оператор_1
    оператор_2
    ...
    оператор_N
```

Условие может быть любым выражением, возвращающим булево значение (`True` или `False`). Если условие истинно — тело цикла выполняется; если ложно — цикл завершается. Пример:

```
count = 0
while count < 5:
    print(count)
    count += 1
# Вывод: 0 1 2 3 4
```

Цикл `for` в Python — синтаксический сахар над итерируемыми объектами (списки, строки, диапазоны и т.д.). Его реализация строится на `while`, но абстрагирует детали.

```
for item in [1, 2, 3]:
    print(item)
# Эквивалентно:
iterable = [1, 2, 3]
iterator = iter(iterable)
while True:
    try:
        item = next(iterator)
        print(item)
    except StopIteration:
        break
```

Концепция:
`for` удобен для перебора элементов коллекции; `while` — когда количество итераций заранее неизвестно и зависит от динамического условия.
Паттерн:
Используйте `while`, если логика цикла опирается на изменение состояния вне коллекции (например, счётчик, флаг, внешний API). Используйте `for`, если перебираете данные напрямую.

Цикл `while` в Python — это цикл с предусловием, где тело выполняется только при истинности условия. После каждой итерации условие проверяется заново. Если после выполнения тела цикла условие остаётся истинным, следующая итерация начинается немедленно. Это означает, что **цикл может выполняться бесконечно**, если условие никогда не становится ложным — такое состояние называется зацикливанием (infinite loop).

Пример:

```
x = 5
while x > 0:
    print(x)
    x -= 1  # без этого условия цикл будет бесконечным
```

Концепция: `else` после `while` — это опциональный блок, который выполняется **только тогда**, когда цикл завершился естественным образом (условие стало ложным), но **не срабатывает при выходе через** `break`.

```
i = 0
found = False
while i < 5:
    if i == 3:
        found = True
        break
    i += 1
else:
    print("Цикл завершился без break")
# Вывод: цикл завершился без break (если break не был вызван)
```

Паттерн: Используйте `else` после `while`, чтобы выполнить код **только при успешном завершении цикла**, а не при его прерывании. Это полезно для реализации логики "попробовать и только потом что-то сделать", например, поиск элемента в списке или ожидание события.

Если после первого же оператора значение `x` становится отрицательным,
цикл **не** будет продолжать выполнение — он завершится немедленно.
Однако если условие цикла остаётся истинным на каждой итерации (например, `while x > 0`), и `break` не вызывается, цикл может выполняться бесконечно.

```
x = 5
while x > 0:
    print(x)
    # x -= 1  # без этого — зацикливание!
```

Концепция:
Цикл `while` проверяет условие **перед каждой итерацией**. Если условие истинно — выполняется тело цикла; если ложно — цикл завершается. Отсутствие изменения состояния внутри тела приводит к бесконечному выполнению.

Паттерн:
Используйте `else` после `while`, чтобы выполнить код **только при успешном завершении цикла**, а не при его прерывании. Это полезно для реализации логики "попробовать и только потом что-то сделать", например, поиск элемента в списке или ожидание события.

```
x = 5
found = False
while x > 0:
    if x == 3:
        found = True
        break
    x -= 1
else:
    print("Цикл завершился без break — элемент не найден")
```

Инструкция управления циклом: `break`

Оператор `break` позволяет **немедленно выйти из цикла**, минуя остаток тела и блок `else`. Он используется при обнаружении нужного условия (например, в поиске или обработке исключений).

```
while True:
    user_input = input("Введите 'quit': ")
    if user_input == "quit":
        break  # выход без проверки else
print("Цикл прерван")
```

Концепция: Вложенные циклы (`nested loops`)

Внутри одного цикла `while` может находиться другой цикл. Это используется, например, для перебора матриц, графов или многоуровневых структур данных.

```
i = 3
while i > 0:
    j = 2
    while j > 0:
        print(f"i={i}, j={j}")
        j -= 1
    i -= 1
```

Вложенные циклы структура и сложность O n m

Рекомендация: Избегайте глубоких вложенности (более 2–3 уровней) — это снижает читаемость и увеличивает сложность отладки.

Синтаксис сокращённого присваивания (`augmented assignment`)

Python поддерживает укороченные операторы изменения значений: `+=`, `-=` и др.

```
x = 5
x += 1   # эквивалентно x = x + 1
y -= 2   # y = y - 2
z *= 3   # z = z * 3
```

Применение:

Удобен при циклическом обновлении счётчиков, индексов или накопителей. Снижает количество строк и уменьшает вероятность ошибок.

На каждой итерации цикла `while` условие истинно — выполнение продолжается до тех пор, пока оно не становится ложным. После выхода из цикла может выполняться блок кода, который **иногда** (как у ящерицы) отброшен и не выполняется. У цикла `while` есть тело, которое выполняется на каждой итерации, и опциональный блок после цикла — он срабатывает только при выходе по условию.

Концепция: Вложенные циклы
В программировании часто возникает ситуация, когда внутри одного цикла находится другой — это **вложенные циклы**. Например: внешний цикл обрабатывает элементы списка, а внутренний — перебирает подэлементы каждого из них.

Пример структуры:

```
while x > 0:
    y = x
    while y > 0:
        print(y)
        y -= 1
    x -= 1
```

Алгоритм: Вложенные циклы — механизм итерирования по двум уровням данных
Количество итераций может расти экспоненциально: если внешний цикл делает `n` шагов, а внутренний — `m`, то общее число операций — $O(n \cdot m)$. Избегайте вложенности более чем на два уровня — это снижает читаемость и увеличивает сложность отладки.

Паттерн: Иерархическая обработка данных
Используется при: - переборе матриц (`for i in range(h): for j in range(w)`), - рекурсивной обработке деревьев, - итерации по словарям с вложенными структурами.

Синтаксис сокращённых операторов присваивания в Python
Python поддерживает **оператор присваивания с модификацией значения**: `+=`, `-=`, `*=`, `/=`, `//=`, `%=`, `**=`, `&=`, `|=`, `^=`, `>>=`, `<<=`.

Пример:

```
x = 5
x += 1   # эквивалентно x = x + 1
y -= 2   # y = y - 2
z *= 3   # z = z * 3
```

Применение: Удобен при циклическом обновлении счётчиков, индексов или накопителей. Снижает количество строк и уменьшает вероятность ошибок.

Концепция: `if` и `else` — ветвление выполнения
Условие в `if`: `if условие:` — блок выполняется, если условие истинно.
`else`: опциональный блок, который выполняется **только при ложном условии**, после завершения проверки условия один раз.

Синтаксис:

```
if x > 0:
    print("Положительное")
else:
    print("Неположительное или ноль")
```

Важно: В `else` не нужно писать `not условие` — это избыточно и снижает читаемость.

Концепция: Цикл `for` в Python — итерирование по коллекциям
Цикл `for` в Python работает **не по счётчику**, а по элементам итерабельного объекта (список, строка, словарь, генератор).

Синтаксис:

```
for item in collection:
    print(item)
```

Пример:

```
numbers = [1, 2, 3]
for num in numbers:
    print(num * 2)
# Вывод: 2, 4, 6
```

Особенность: `for` не требует явного управления индексами — он автоматически проходит по всем элементам. Это делает код короче и безопаснее (нет ошибок выхода за границы).

Сравнение с `while`: - `while` — цикл по условию. - `for` — цикл по итерируемому объекту.

Пример: вывод чисел от x до 0 в каждом вложенном цикле

```
x = int(input("Введите число: "))
while x > 0:
    y = x
    while y > 0:
        print(y)
        y -= 1
    x -= 1
print("стоп")
```

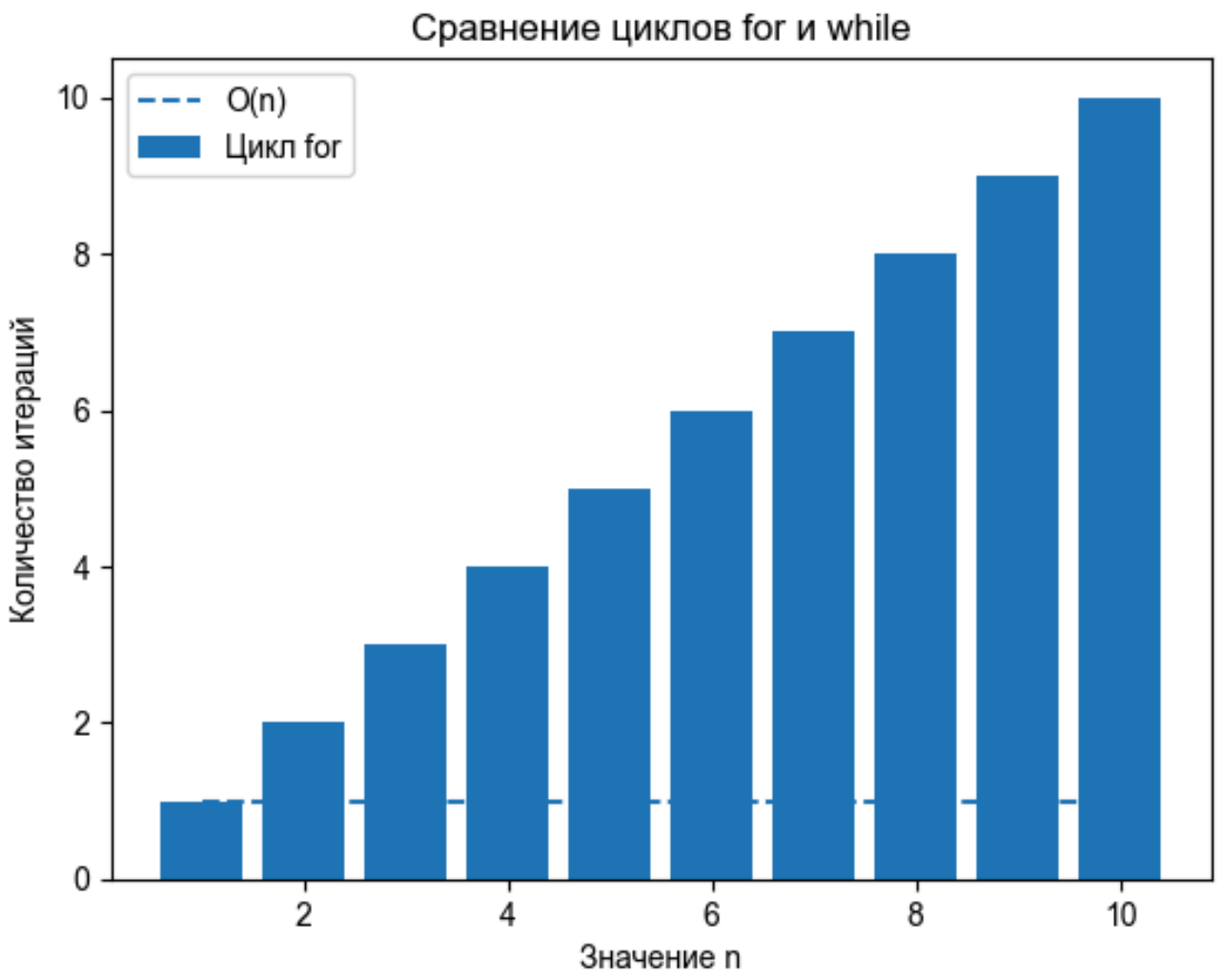
Результат при вводе `3`:

```
3
2
1
2
1
1
стоп
```

> Примечание: порядок вывода зависит от расположения инструкций. Если `x -= 1` перед внутренним циклом — вывод будет: `3,2,1,2,1,1`.
> Если после — сначала выведутся все значения для текущего `x`, затем `x` уменьшится.

Вывод:

- `while` — цикл по условию.
- `for` — цикл по итерируемому объекту («волшебство» Python

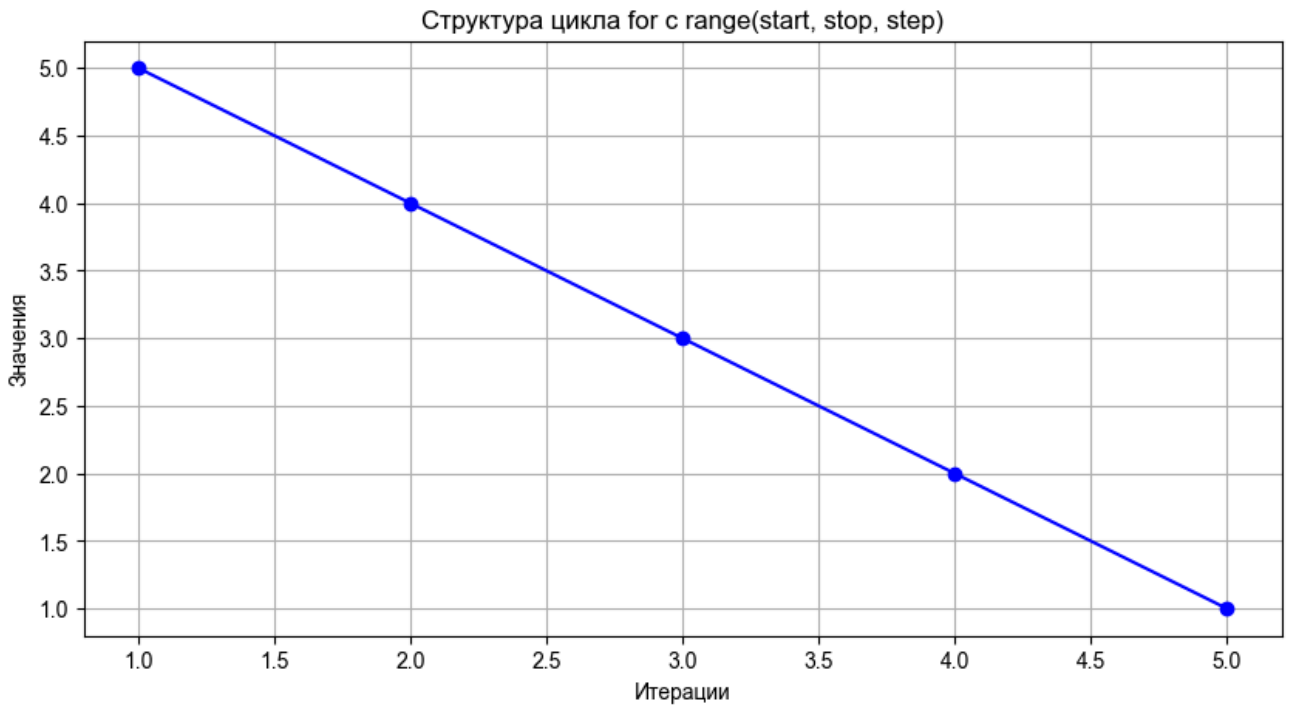


Сравнение циклов for и while

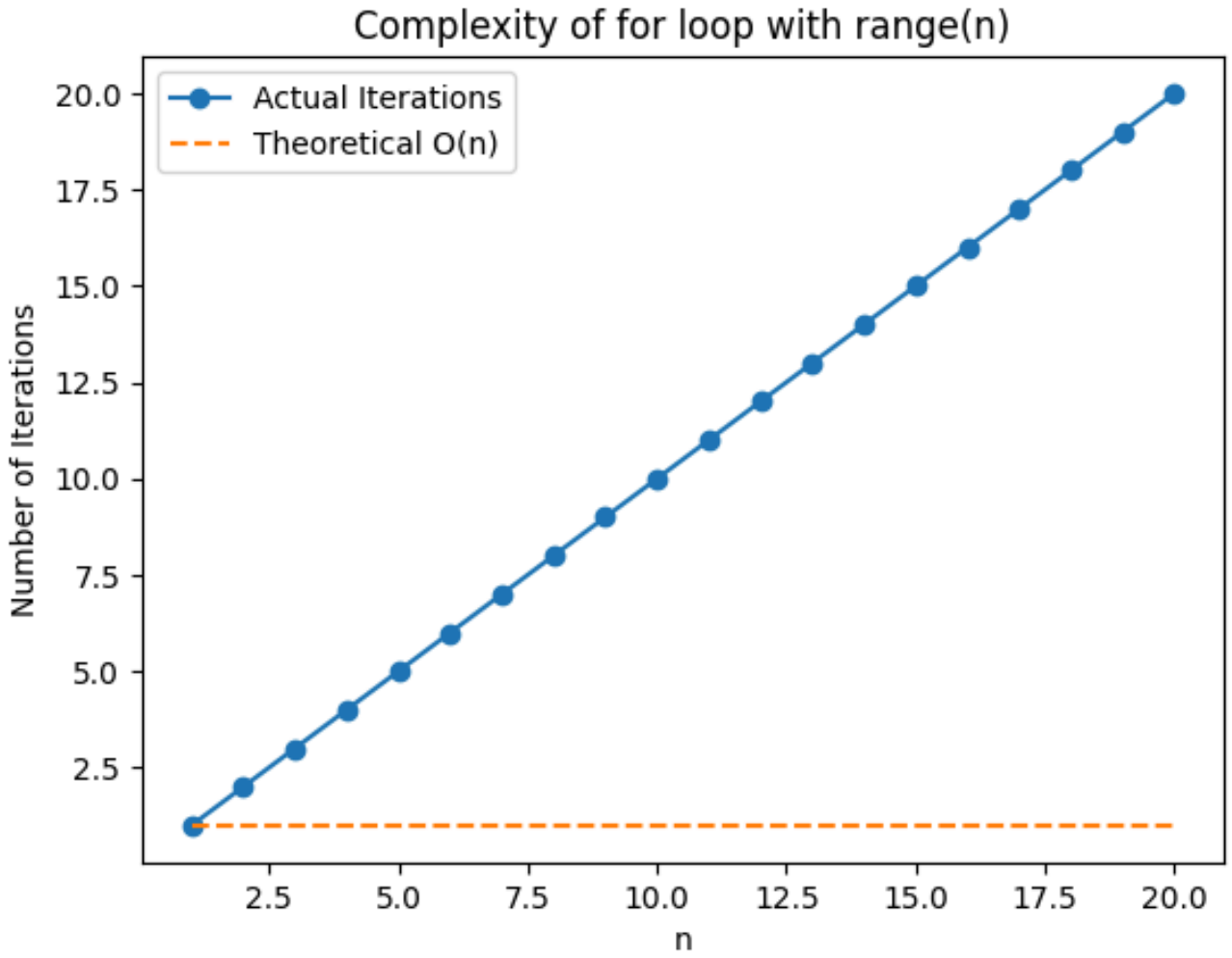
).
- Вложенные циклы возможны, но не рекомендуется более двух уровней.
- Сокращённые операторы присваивания (`+=` , `-=` и др.) повышают читаемость и безопасность.
- Блок `else` после `while` / `for` выполняется **только при выходе по условию**, а не по `break` .

x = -1
for i in range(1, 10, 1):
print(i)

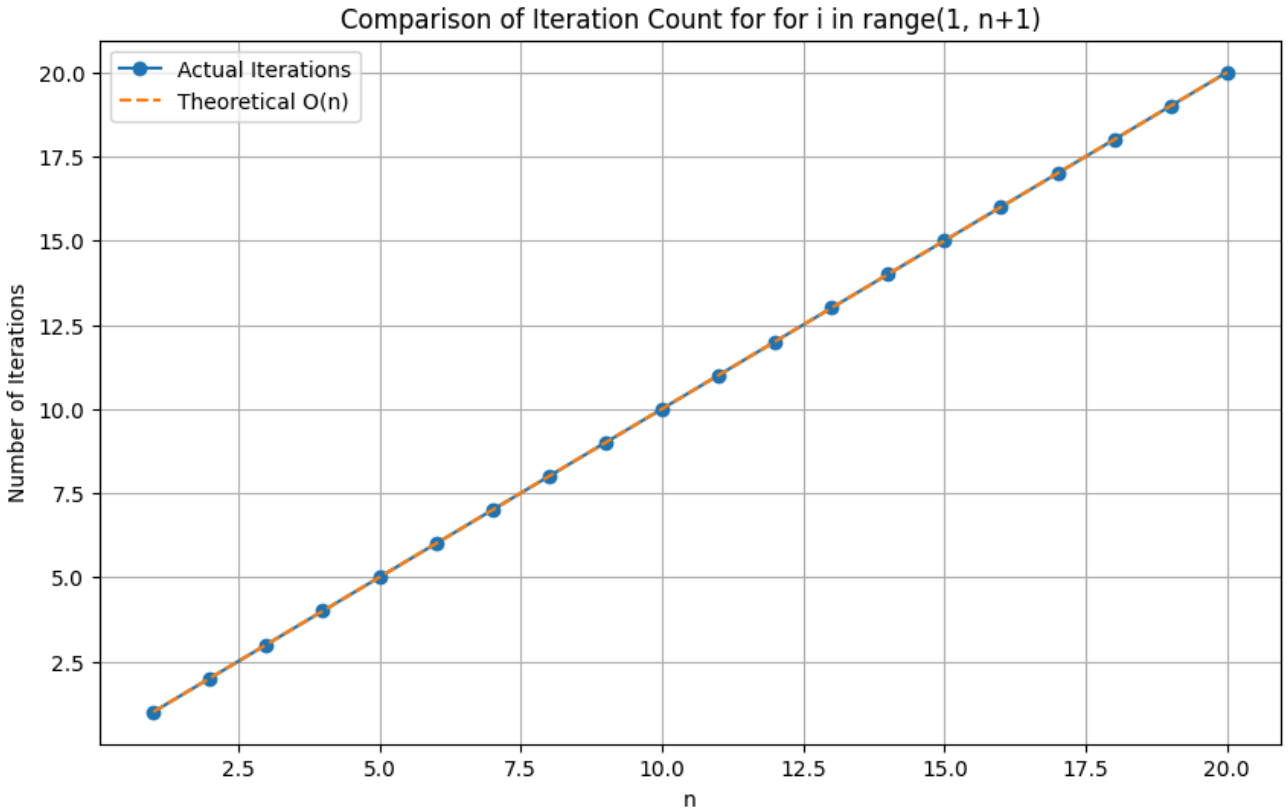
— цикл по условию.
- `for` — цикл по итерируемому объекту («волшебство» Python).
- Вложенные циклы возможны, но не рекомендуется более двух уровней



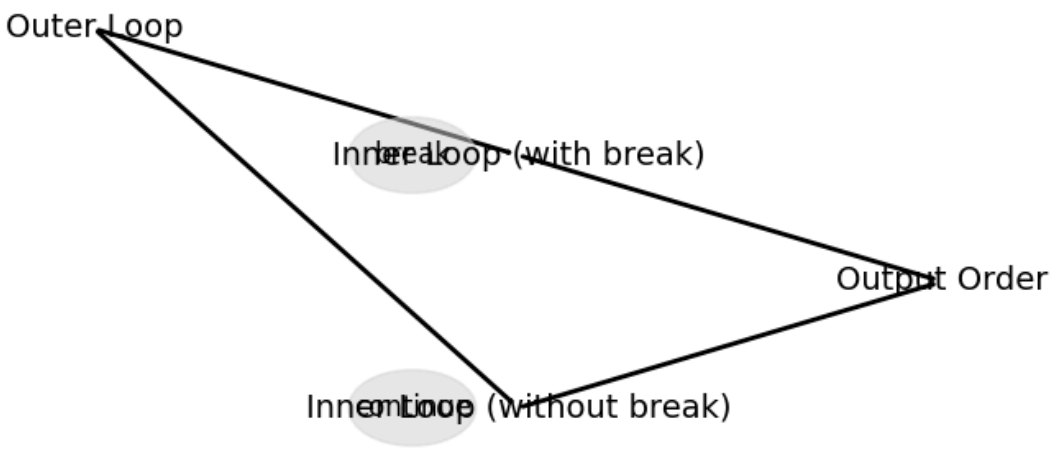
Структура цикла for с range start stop step



Сложность цикла for с range n



Сравнение поведения блока else в for while



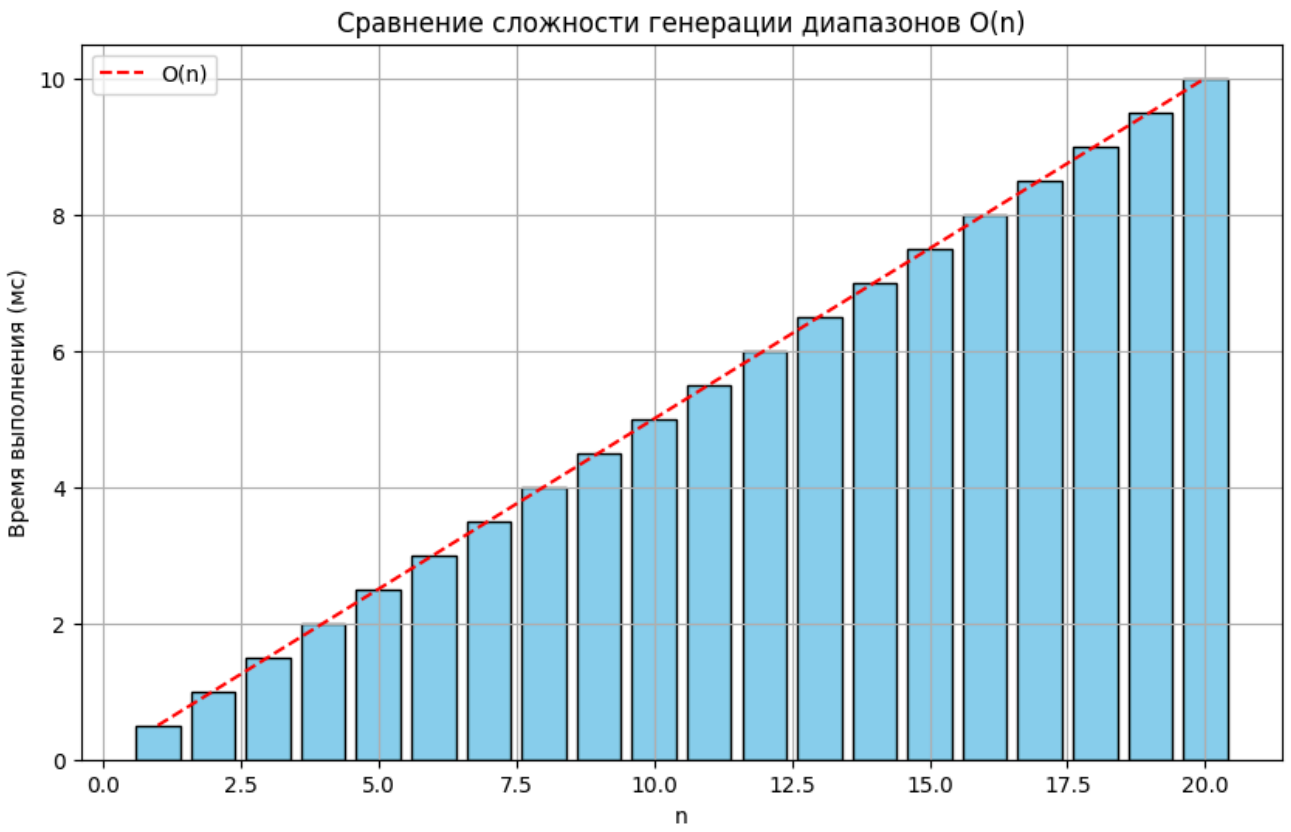
Архитектура вложенных циклов

.
- Сокращённые операторы присваивания (`+=`, `-=` и др.) повышают читаемость и безопасность.
- Блок `else` после `while` / `for` выполняется **только при выходе по условию**, а не по `break`.

```
x = 3
y = x - 1
print(y)  # вывод: 2

# цикл for с диапазоном range(start, stop, step)
for i in range(1, 10, 1):
    print(i)

# если шаг отрицательный и старт > стоп — пустая последовательность
for i in range(5, 1, -1):
    print(i)  # выводит: 5, 4, 3, 2

# break внутри цикла останавливает выполнение, else не срабатывает
for i in range(3):
    if i == 1:
        break
else:
    print("else")  # не выполнится

# else выполняется только при естеочном завершении цикла
for i in range(3):
    pass
else:
    print("цикл завершён без break")  # выполнится
```

Можно использовать `else` в цикле `for`, который срабатывает только при естественном завершении цикла — без `break`. Если `break` присутствует, блок `else` пропускается. Это работает и для вложенных циклов: `for` внутри `while`, `while` внутри `for` — допустимо.

Генератор `range` — универсальный инструмент создания арифметической прогрессии. Его поведение определяется тремя параметрами по аббревиатуре SSS: **Start** (начало), **Stop** (конец), **Step** (шаг



Сравнение сложности генерации диапазонов O n

).
- Диапазон `range(1, 10, 1)` включает числа от 1 до 9 включительно — стоп не входит в результат.
- Математически: $[start, stop) \cap \mathbb{Z}$, где $\mathbb{Z}$ — множество целых чисел.

Если шаг отрицательный (`step < 0`), а `stop > start`, последовательность пустая — цикл не выполнится ни разу. Аналогично, если `start > stop` и `step > 0`, результат также пустой. Это корректное поведение: попытка построить невозрастающую или неубывающую прогрессию в неверном направлении приводит к отсутствию элементов.

Пример:

```
range(1, 10, 2)  # → [1, 3, 5, 7, 9]
```

Число `10` не включается — оно строго меньше стопа.

Для обратного диапазона (убывание):
- `range(10, 1, -1)` → `[10, 9, ..., 2]`, но **не включает** `1`.
- Чтобы включить `1`, нужно: `range(10, 0, -1)` → `[10, 9, ..., 1]`.

Концепция: `continue` — оператор пропуска текущей итерации. Он "телепортирует" выполнение к началу цикла, минуя остаток тела. Полезен для фильтрации или пропуска условий без выхода из цикла.
Пример:

```
for i in range(5):
    if i % 2 == 0:
        continue
    print(i)  # выведет: 1, 3
```

Алгоритм: `range` — эффективный генератор последовательностей за $O(n)$ времени и $O(1)$ памяти (ленивый). Поддерживает отрицательные шаги, проверку границ и пустые случаи.
Паттерн: Использование `continue` вместо вложенных условий улучшает читаемость кода при частичном пропуске элементов.